

An aerial photograph of the University of British Columbia Okanagan campus. The image shows several large, modern brick buildings with flat roofs, some featuring solar panels. The campus is surrounded by lush green trees and lawns. In the background, there are rolling hills and mountains under a clear blue sky with a few wispy clouds. The lighting suggests it's either early morning or late afternoon, with long shadows cast across the landscape.

# Special Topics: Applied Computer Vision and Deep Learning

UBCO Master of Data Science – DATA 589



# Topics

---

- Self-Supervised Learning
- MAE : Mask Auto Encoders
- Dino : Self-Distillation with No Labels
- CLIP: Contrastive Language Image Pre-training

# Self-Supervised Learning

# Self-Supervised Learning

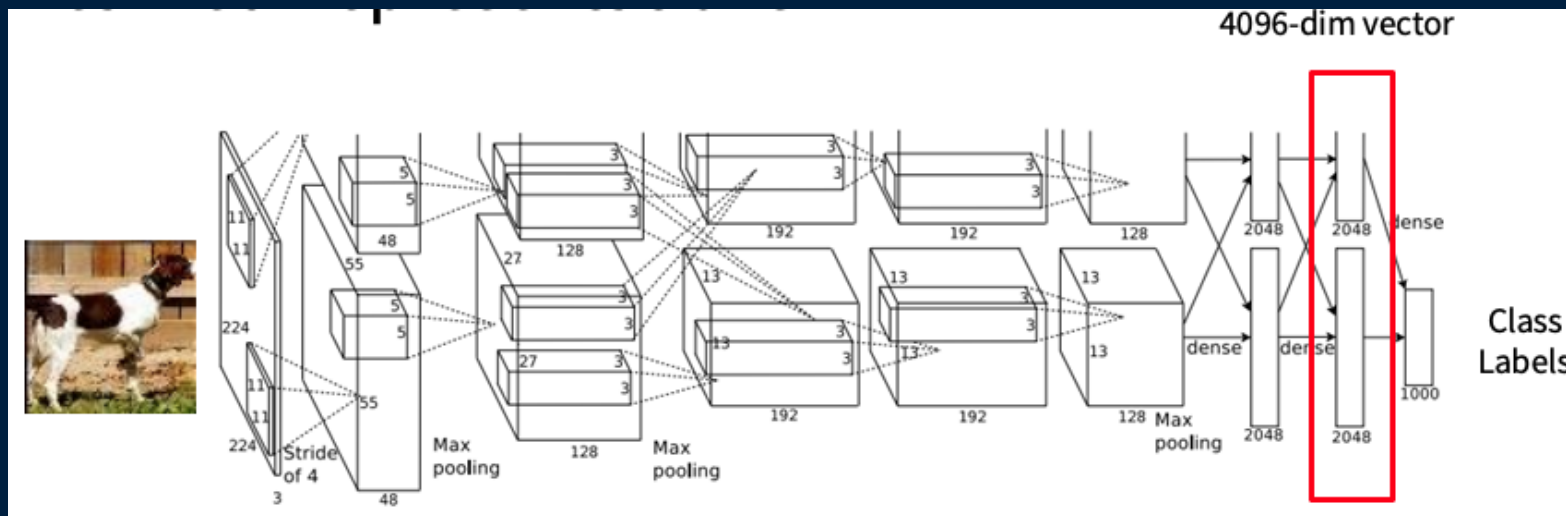
---

- In **supervised learning**, humans provide the target
- In **self-supervised learning**, the **data provides the target**
- In **unsupervised learning**, there may be **no explicit target at all**

**Self-supervised** learning is a modern training strategy where we turn unlabeled data into a supervised-style task by generating targets from the data itself.

That is why models like **DINO**, **MAE**, and **CLIP-style pretraining** are often discussed under self-supervised learning.

# Learned Representations



What is the problem with large-scale training?

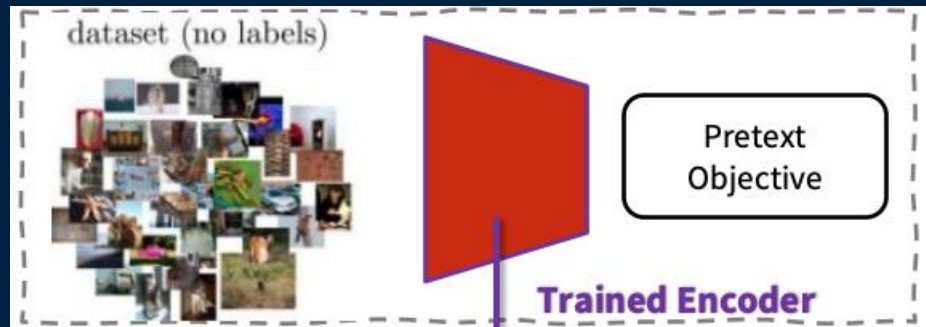
We need a lot of labeled data

Is there a way we can train neural networks without the need for huge manually labeled datasets?

# Self-Supervised Learning

**Pretext Task** - Define a task based on the data itself

- No manual annotation - Could be considered an unsupervised task; but we learn with supervised learning objectives, e.g., classification or regression.





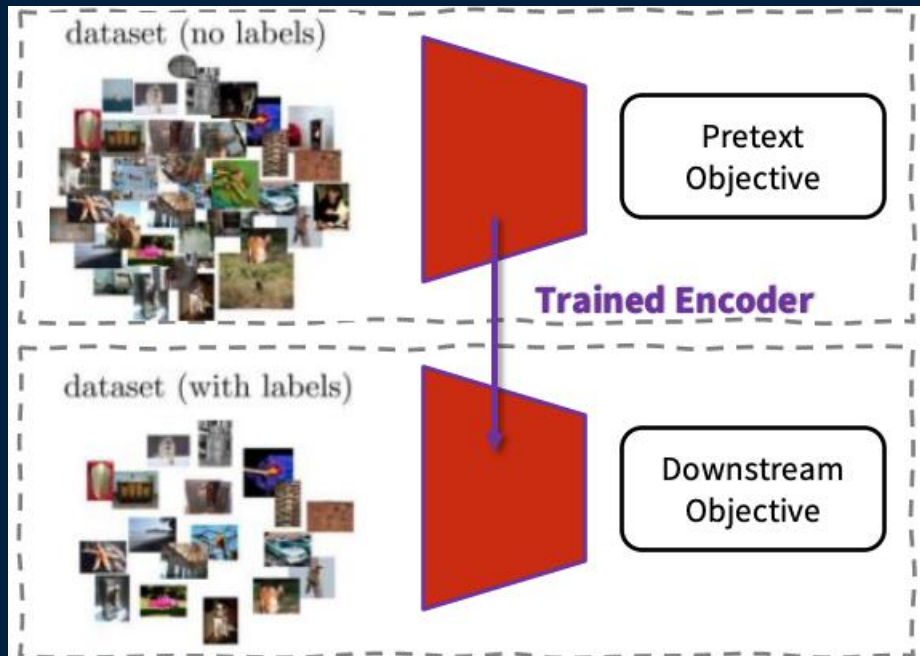
# Self-Supervised Learning

**Pretext Task** - Define a task based on the data itself

- No manual annotation - Could be considered an unsupervised task;
- but we learn with supervised learning objectives, e.g., classification or regression.

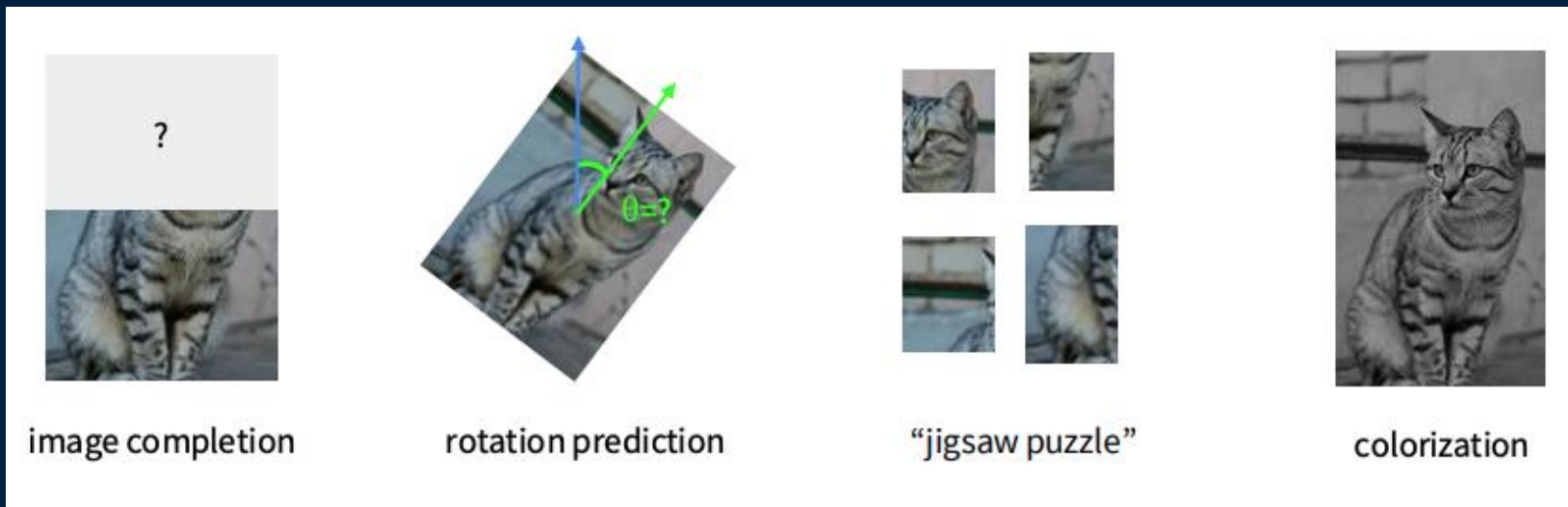
**Downstream Task** - The application you care about

- You do not have large datasets
- The dataset is labeled



# Pretext Tasks

Example: learn to predict image transformations / complete corrupted images



1. Solving the pretext tasks allow the model to learn good features.
2. We can automatically generate labels for the pretext tasks.



# Common Pretext Tasks in Vision

---

- **Masked image prediction**

Predict missing patches from the visible ones

- **Contrastive learning**

Bring two views of the same image closer, push others apart

- **Image rotation prediction**

Predict how the image was rotated

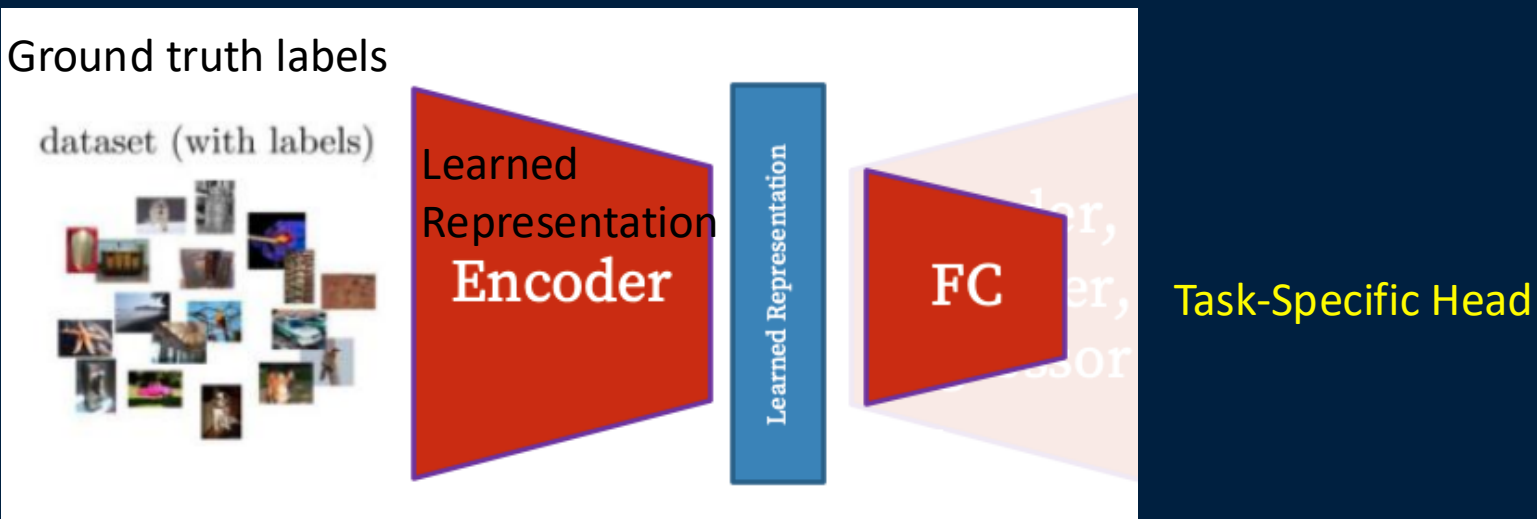
- **Patch or region matching**

Learn whether two crops come from the same image

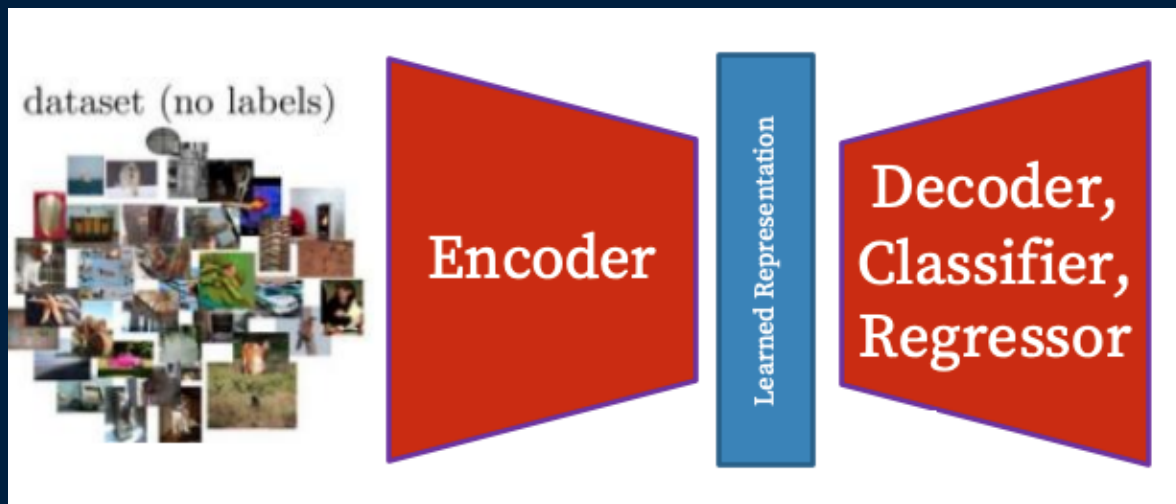
- **Teacher–student consistency**

Match stable representations across views

# Self-Supervised Learning – Downstream Task



# Self-Supervised Learning – Pretext Task



Labels  
automatically  
generated from  
data

# Evaluation of a Self-Supervised Learning

Does the learned representation help on real downstream tasks?

## Downstream performance

- Linear probe or fine-tune on classification, detection, segmentation

## Representation quality

- Clustering, nearest neighbors, t-SNE/PCA visualization

## Generalization

- Transfer to new datasets, robustness to perturbations

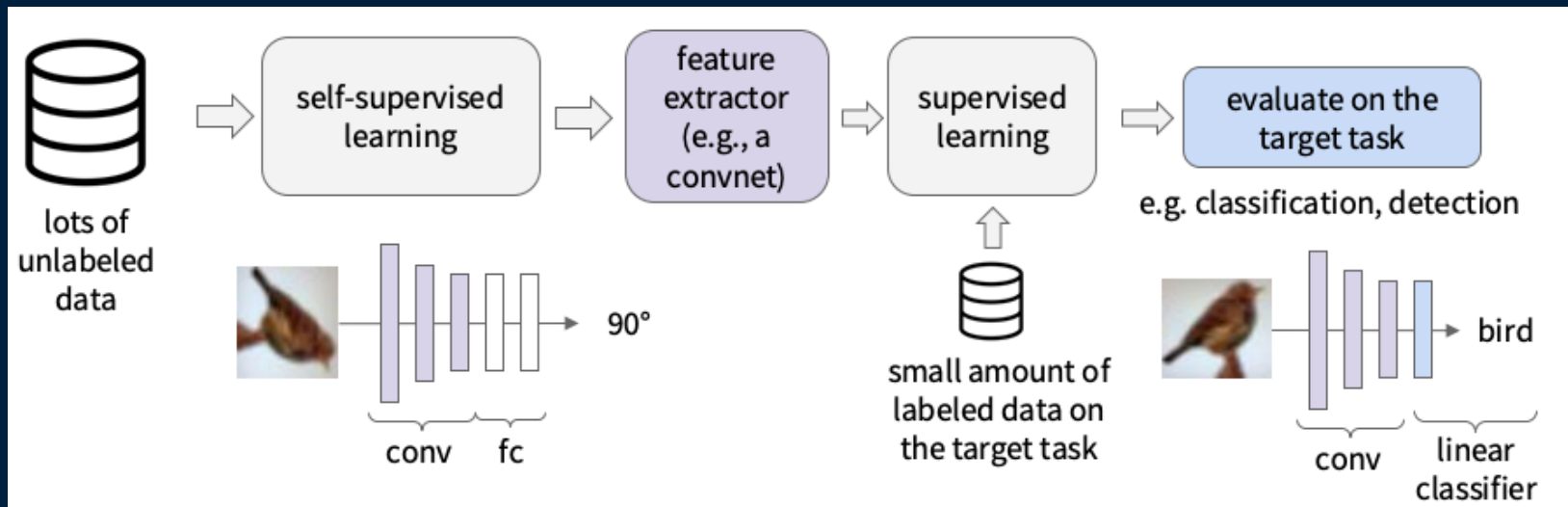
## Practical efficiency

- Training cost, inference cost, data and compute requirements

## Key takeaway:

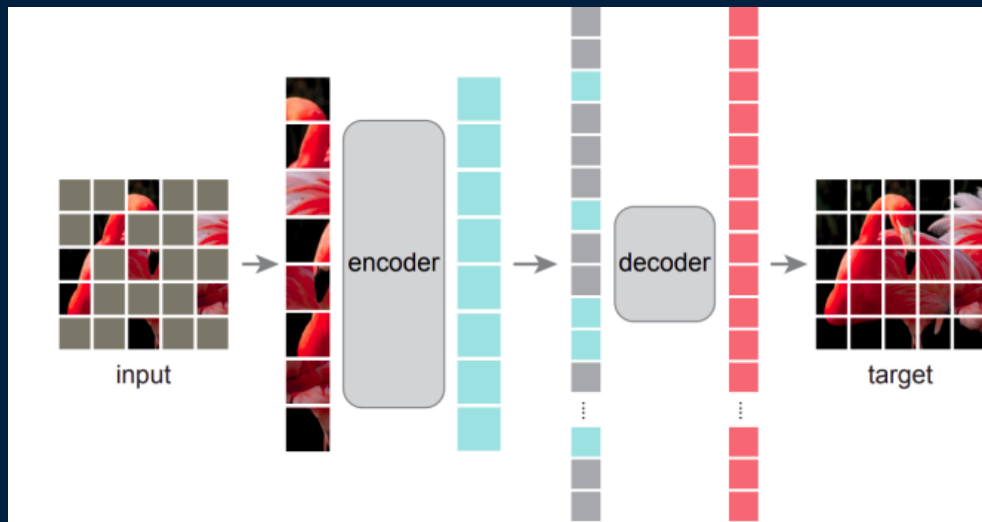
A good self-supervised method learns representations that transfer well.

# Evaluation of a Self-Supervised Learning



A model could recognize the correct rotation of an object only if it has the “visual commonsense” of what the object should look like.

# Masked Auto Encoders (MAE)



- A self-supervised method for learning visual representations
- Randomly **mask a large portion of image patches**
- Feed only the **visible patches** into the encoder
- Use a decoder to **reconstruct the missing patches**
- Train the model by minimizing **reconstruction error**

# Why MAE Matters

---

- Learns from **unlabeled images**
- Makes pretraining more efficient by encoding only **visible patches**
- Encourages the model to understand **global structure and context**
- Produces strong pretrained representations for:
  - classification**
  - detection**
  - segmentation**
- A major foundation-model approach for vision



# Train a Pre-trained Model

---

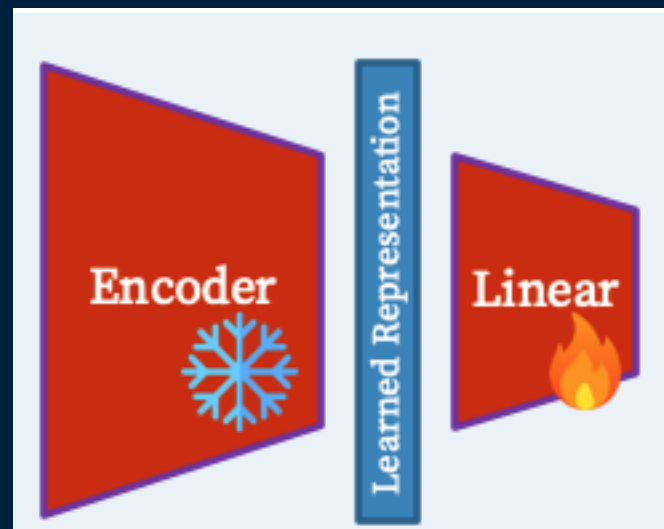
How to train a model on downstream tasks?

**1- Linear Probing**

**2- Full fine tuning**

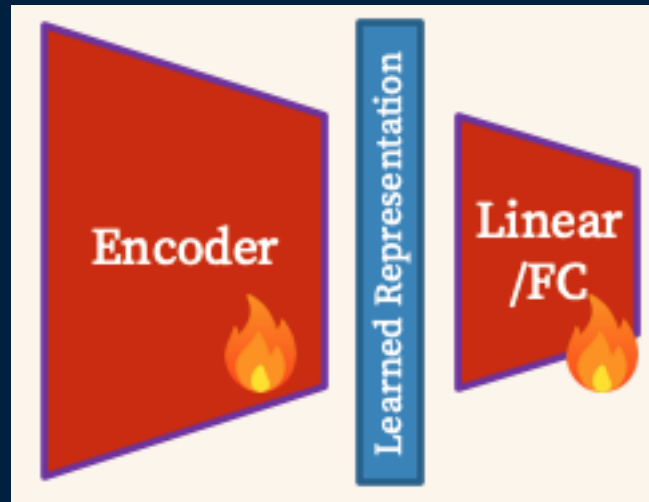
# Linear Probing

- Start with a **pretrained encoder**
- **Freeze** the encoder weights
- Add a **single linear classifier** on top
- Train only the linear layer using labeled data
- Used to measure the quality of the learned representation



# Full Fine-tuning

- Start with a **pretrained encoder**
- Add a task head such as:
  - linear layer
  - MLP
  - classifier / detector head
- **Update the encoder weights** during training
- Train on labeled downstream data
- Usually gives higher task-specific performance than linear probing



# Question

---

Question: **Which statement best describes self-supervised learning?**

- A. The model always requires a large labeled dataset
- B. The model learns only from random noise
- C. The data itself provides the target for learning
- D. The model can only be used for classification
- E. The model must use text captions as labels

# Question

---

**Question: In MAE, what is the model mainly trained to do during pretraining?**

- A. Predict bounding boxes for objects
- B. Reconstruct missing image patches from visible patches
- C. Match image embeddings to text captions
- D. Classify images into fixed labeled categories
- E. Detect unseen classes with no retraining

# Question

---

Question: **What is the main purpose of linear probing after self-supervised pretraining?**

- A. To generate captions for unlabeled images
- B. To evaluate whether the pretrained encoder learned useful features
- C. To replace the encoder with a larger model
- D. To train the model from scratch on a new dataset
- E. To create new image augmentations for pretraining

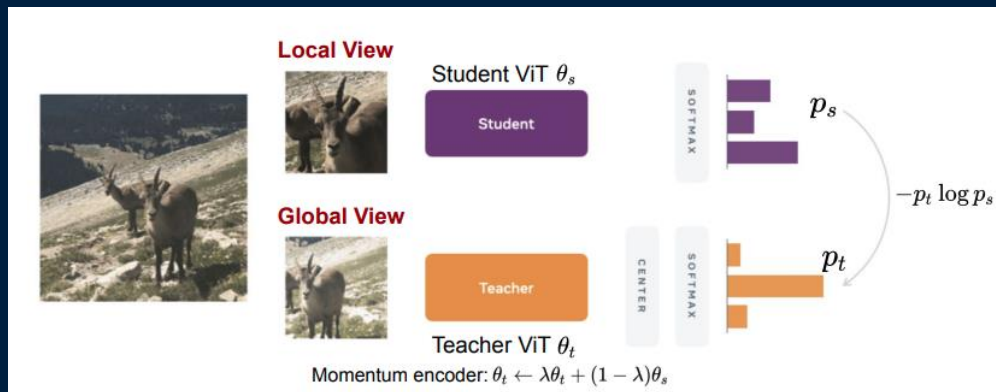
# Dino: Self-Distillation with No Labels



Self-attention from a Vision Transformer with  $8 \times 8$  patches trained with no supervision.

# Dino: Teacher–Student Learning

- Use two copies of a model:
- **Student** = the model we actively train
- **Teacher** = the model that provides the target output
- Give them related versions of the same image
- The **student learns to match the teacher's prediction**
- The **teacher is updated more slowly** to stay stable
- This creates a learning signal **without manual labels**





# Dino – How it works

---

- Start with one image and create **multiple augmented crops**
- Send crops to both a **student** and a **teacher** network
- The **student** is trained to match the teacher's output distribution
- The **teacher is not trained by backpropagation**
- The teacher is updated using **EMA** (*exponential moving average of the student weights*)
- Result: the encoder learns strong semantic features

# Dino

---

## Important newer branches

### **DINOv2**

Stronger large-scale visual pretraining with robust transferable features

### **Grounding DINO**

Extends DINO-style detection with language for **open-set / open-vocabulary object detection**

### **Grounding DINO 1.5**

Improved open-set detection, including a more efficient edge-oriented version

### **DINO-X**

A newer open-world object-centric model supporting detection, segmentation, pose, captioning, and more

# What is a Foundation Model?

A **foundation model** is a model trained on **large-scale data** so it can learn **general-purpose representations**

It is called “foundation” because it can serve as a **base model** for many different downstream tasks

Instead of training a new model from scratch for every task, we **adapt** the same pretrained model

Foundation models are usually:

- trained on very large datasets
- reusable across tasks
- transferable to new domains
- adaptable with fine-tuning or prompting

Examples in vision:

- DINOv2
- CLIP
- SAM
- large ViT backbones

Key idea:

A foundation model is not built for only one task — it becomes a strong starting point for many tasks.

# From Self-Supervised Vision to Vision–Language Learning

---

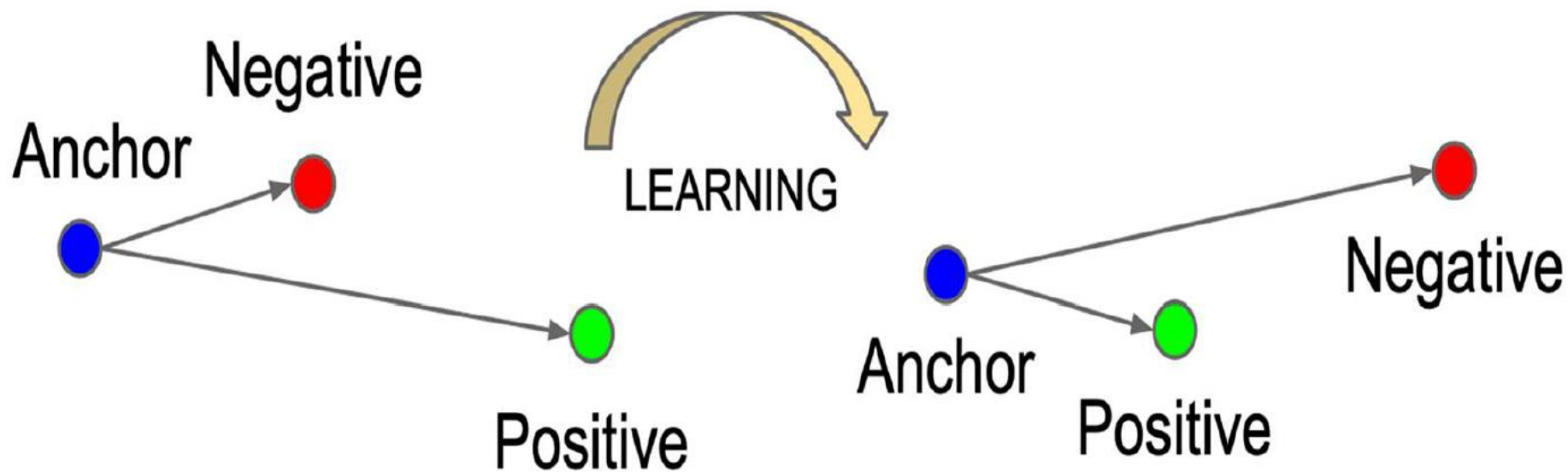
## **Vision Language Model: CLIP** **Contrastive Language Image Pre-training (CLIP)**

# CLIP

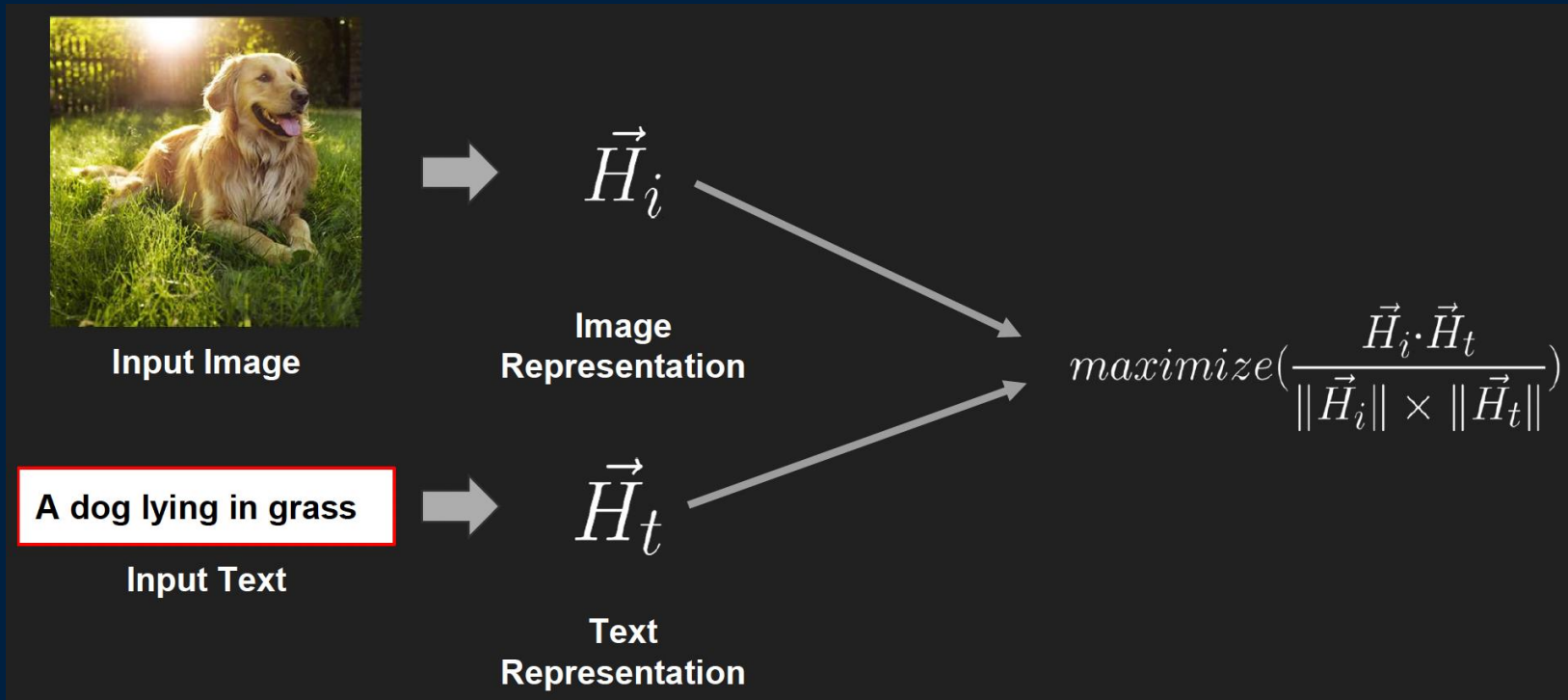
---

- **DINO** learns visual representations from **images alone**
- It is a **pure self-supervised vision method**
- The model learns by making different views of the **same image** produce similar embeddings
- **CLIP** takes the next step: it learns from **image–text pairs**
- Instead of learning only from visual consistency, it learns to **align images with language**
- This moves us from **pure self-supervised vision** toward **vision–language pretraining**

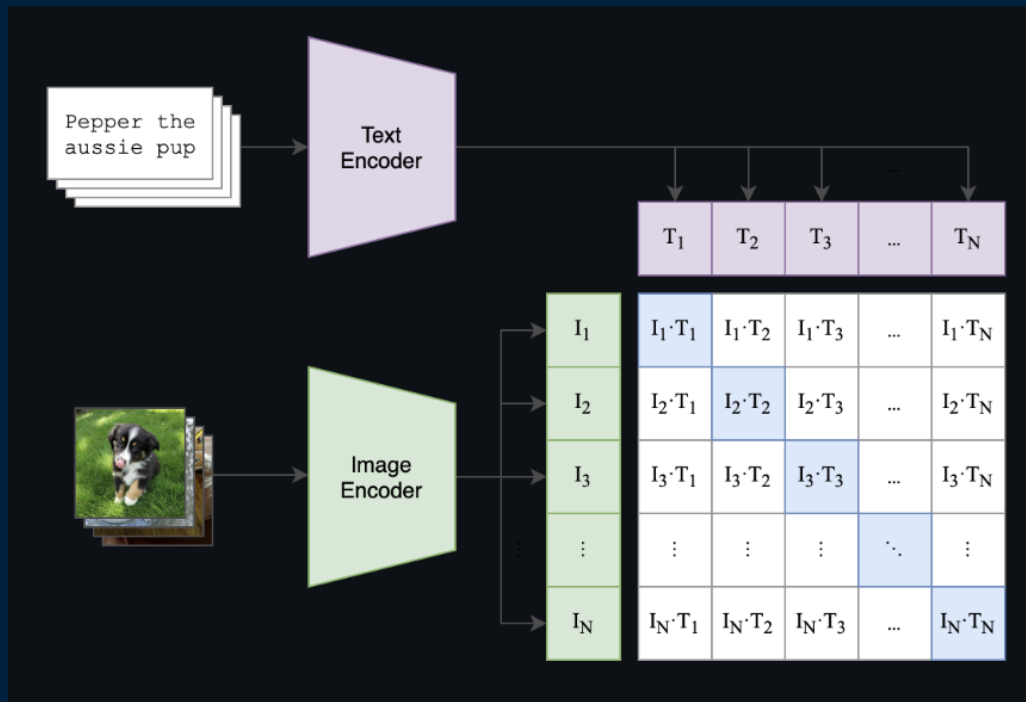
# What is Contrastive Learning



# Image-Text Contrastive Learning



# Contrastive pre-training



- Encode batch of text samples
- Encode batch of image samples
- Maximize cosine similarity between correct matches



# Compute CLIP Loss

|          | $T_1$           | $T_2$           | $T_3$           | ...      | $T_N$           |
|----------|-----------------|-----------------|-----------------|----------|-----------------|
| $I_1$    | $I_1 \cdot T_1$ | $I_1 \cdot T_2$ | $I_1 \cdot T_3$ | ...      | $I_1 \cdot T_N$ |
| $I_2$    | $I_2 \cdot T_1$ | $I_2 \cdot T_2$ | $I_2 \cdot T_3$ | ...      | $I_2 \cdot T_N$ |
| $I_3$    | $I_3 \cdot T_1$ | $I_3 \cdot T_2$ | $I_3 \cdot T_3$ | ...      | $I_3 \cdot T_N$ |
| $\vdots$ | $\vdots$        | $\vdots$        | $\vdots$        | $\ddots$ | $\vdots$        |
| $I_N$    | $I_N \cdot T_1$ | $I_N \cdot T_2$ | $I_N \cdot T_3$ | ...      | $I_N \cdot T_N$ |

## Step 1 — Build the Similarity Matrix

In one batch of size  $N$ :

- We have  $N$  image embeddings:

$$I_1, I_2, \dots, I_N$$

- We have  $N$  text embeddings:

$$T_1, T_2, \dots, T_N$$

- We compute **all pairwise cosine similarities**:

$$S_{ij} = I_i \cdot T_j$$

This creates an  $N \times N$  **similarity matrix**.

**Interpretation:**

Each row  $\rightarrow$  one image compared to all texts

Each column  $\rightarrow$  one text compared to all images

Diagonal elements  $\rightarrow$  correct image-text pairs

# Compute CLIP Loss

## Step 2 — Convert Similarities to Probabilities

For each image  $I_i$ :

We take row  $i$  of the matrix:

$$y_i^m = [S_{i1}, S_{i2}, \dots, S_{iN}]$$

Apply **softmax** over the row.

This gives a probability distribution over captions:

Which caption matches this image?

Ground truth:

$$m_i = \text{one-hot vector}$$

Only the diagonal position is 1.

|          | $T_1$           | $T_2$           | $T_3$           | ...      | $T_N$           |
|----------|-----------------|-----------------|-----------------|----------|-----------------|
| $I_1$    | $I_1 \cdot T_1$ | $I_1 \cdot T_2$ | $I_1 \cdot T_3$ | ...      | $I_1 \cdot T_N$ |
| $I_2$    | $I_2 \cdot T_1$ | $I_2 \cdot T_2$ | $I_2 \cdot T_3$ | ...      | $I_2 \cdot T_N$ |
| $I_3$    | $I_3 \cdot T_1$ | $I_3 \cdot T_2$ | $I_3 \cdot T_3$ | ...      | $I_3 \cdot T_N$ |
| $\vdots$ | $\vdots$        | $\vdots$        | $\vdots$        | $\ddots$ | $\vdots$        |
| $I_N$    | $I_N \cdot T_1$ | $I_N \cdot T_2$ | $I_N \cdot T_3$ | ...      | $I_N \cdot T_N$ |

# Compute CLIP Loss

|          | $T_1$           | $T_2$           | $T_3$           | ...      | $T_N$           |
|----------|-----------------|-----------------|-----------------|----------|-----------------|
| $I_1$    | $I_1 \cdot T_1$ | $I_1 \cdot T_2$ | $I_1 \cdot T_3$ | ...      | $I_1 \cdot T_N$ |
| $I_2$    | $I_2 \cdot T_1$ | $I_2 \cdot T_2$ | $I_2 \cdot T_3$ | ...      | $I_2 \cdot T_N$ |
| $I_3$    | $I_3 \cdot T_1$ | $I_3 \cdot T_2$ | $I_3 \cdot T_3$ | ...      | $I_3 \cdot T_N$ |
| $\vdots$ | $\vdots$        | $\vdots$        | $\vdots$        | $\ddots$ | $\vdots$        |
| $I_N$    | $I_N \cdot T_1$ | $I_N \cdot T_2$ | $I_N \cdot T_3$ | ...      | $I_N \cdot T_N$ |

## Step 3 — Image-to-Text Cross Entropy Loss

$$L_m = \frac{1}{N} \sum_{i=1}^N \phi(y_i^m, m_i)$$

Where:

$\phi$  = cross entropy

$y_i^m$  = predicted distribution

$m_i$  = one-hot correct label

This trains:

Each image should select its correct caption.

# Compute CLIP Loss

|          | $T_1$           | $T_2$           | $T_3$           | ...      | $T_N$           |
|----------|-----------------|-----------------|-----------------|----------|-----------------|
| $I_1$    | $I_1 \cdot T_1$ | $I_1 \cdot T_2$ | $I_1 \cdot T_3$ | ...      | $I_1 \cdot T_N$ |
| $I_2$    | $I_2 \cdot T_1$ | $I_2 \cdot T_2$ | $I_2 \cdot T_3$ | ...      | $I_2 \cdot T_N$ |
| $I_3$    | $I_3 \cdot T_1$ | $I_3 \cdot T_2$ | $I_3 \cdot T_3$ | ...      | $I_3 \cdot T_N$ |
| $\vdots$ | $\vdots$        | $\vdots$        | $\vdots$        | $\ddots$ | $\vdots$        |
| $I_N$    | $I_N \cdot T_1$ | $I_N \cdot T_2$ | $I_N \cdot T_3$ | ...      | $I_N \cdot T_N$ |

## Step 4 — Text-to-Image Direction

Now do the same for columns.

For each text  $T_i$ :

$$y_i^t = [S_{1i}, S_{2i}, \dots, S_{Ni}]$$

Apply softmax over column.

$$L_t = \frac{1}{N} \sum_{i=1}^N \phi(y_i^t, t_i)$$

This trains:

Each caption should select its correct image.

# Compute CLIP Loss

## Final CLIP Loss

|          | $T_1$           | $T_2$           | $T_3$           | ...      | $T_N$           |
|----------|-----------------|-----------------|-----------------|----------|-----------------|
| $I_1$    | $I_1 \cdot T_1$ | $I_1 \cdot T_2$ | $I_1 \cdot T_3$ | ...      | $I_1 \cdot T_N$ |
| $I_2$    | $I_2 \cdot T_1$ | $I_2 \cdot T_2$ | $I_2 \cdot T_3$ | ...      | $I_2 \cdot T_N$ |
| $I_3$    | $I_3 \cdot T_1$ | $I_3 \cdot T_2$ | $I_3 \cdot T_3$ | ...      | $I_3 \cdot T_N$ |
| $\vdots$ | $\vdots$        | $\vdots$        | $\vdots$        | $\ddots$ | $\vdots$        |
| $I_N$    | $I_N \cdot T_1$ | $I_N \cdot T_2$ | $I_N \cdot T_3$ | ...      | $I_N \cdot T_N$ |

$$L = \frac{L_m + L_t}{2}$$

# Zero-Shot Learning & How CLIP Enables It

---

## Zero-shot learning means:

The model can recognize classes it has NEVER been explicitly trained on.

No:

Fine-tuning

Additional training

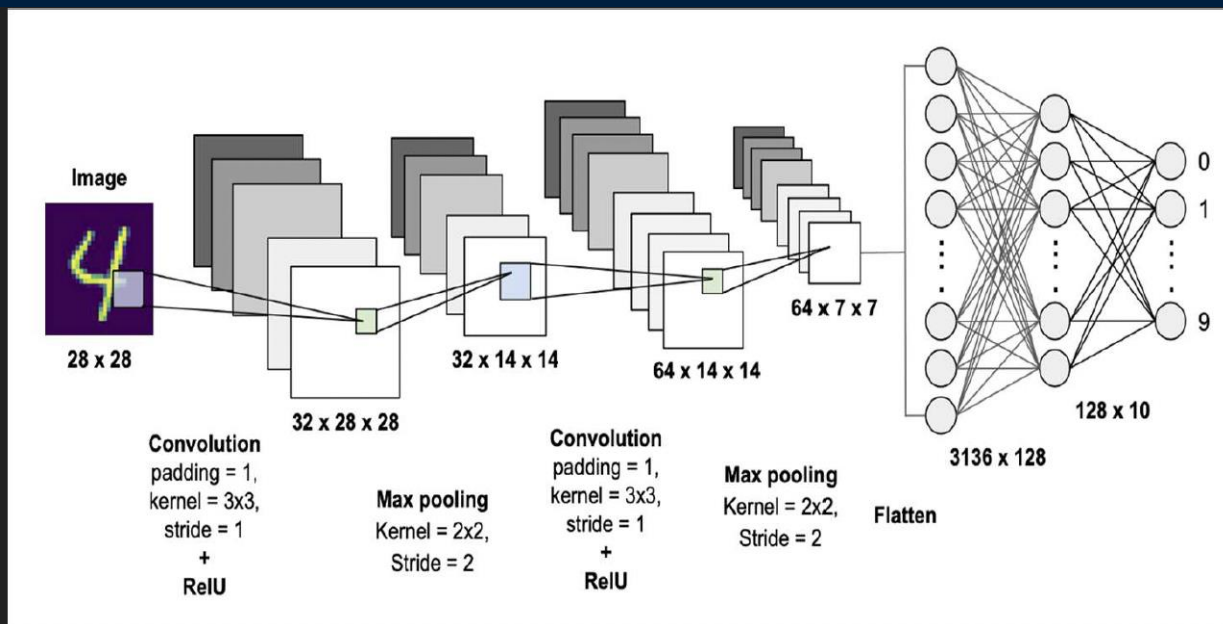
Task-specific classifier

## Traditional supervised learning:

- Train on fixed categories (dog, cat, car)
- Cannot recognize unseen categories
- Zero-shot learning:
- Generalizes to new concepts using language.

# Supervised Classification

supervised models are closed-set.



# Why CLIP Is Different

---

**CLIP does NOT learn a fixed classifier.**

Instead, it learns:

*Image embedding  $\leftrightarrow$  Text embedding*

So classification becomes:

*Which text description is most similar to the image?*

That means:

**You can change the text at inference time.**



# Why this works

---

During training, CLIP learned:

- Millions of image–text pairs
- General visual-language alignment
- Semantic relationships between words and visual patterns

So even if the model never saw:

- “giraffe playing soccer”

It understands:

- giraffe
- soccer
- composition of concepts

# Supervised Learning vs Zero-Shot Learning

## Supervised :

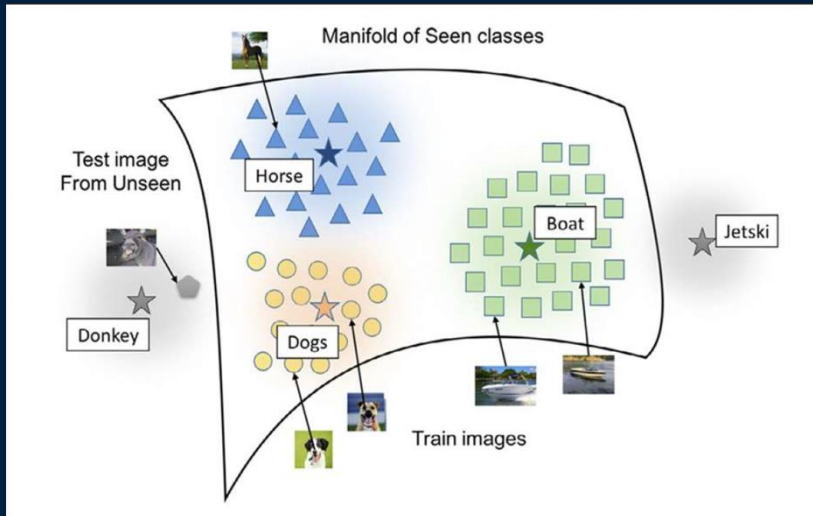
- Requires labeled training data for each class
- Training and testing classes are the same
- Model learns a fixed classifier
- Closed-set recognition
- Adding new classes → retraining required
- Pipeline:

- *Labeled Data → Training → Fixed Classifier*

## Zero-Shot Learning:

- Model generalizes to unseen classes
- Uses semantic information (e.g., language)
- No task-specific retraining required
- Open-vocabulary recognition\
- Classes defined at inference time
- **Pipeline (CLIP-style):**
  - *Pretrained Model + Text Prompts → Prediction*

# What is Zero Shot Learning?



- The model can correctly classify instances from classes it never saw during training.
- No labeled examples of the new class are provided.
- Donkey is semantically related to Horse
- Jetski is semantically related to Boat
- If the model understands semantic relationships, it can place unseen samples near related concepts.
- Zero-shot learning relies on semantic relationships to recognize unseen categories without retraining.

# Limitations of Zero-Shot CLIP

- ◆ Prompt Sensitivity
  - Performance depends on wording
  - Small phrasing changes can affect accuracy
- ◆ Dataset Bias
  - Trained on internet-scale data
  - Inherits cultural and social biases
- ◆ Domain Shift
  - Works well on natural images
  - Struggles in medical, SAR, industrial domains
- ◆ Fine-Grained Recognition
  - Weak on subtle differences
  - Visually similar classes confuse the model
- ◆ Limited Reasoning
  - Based on similarity, not true understanding
  - Weak at complex spatial or logical reasoning

## 1. Linear Probe

**Goal:** Evaluate the quality of learned image features.

**Procedure:**

- Freeze the image encoder
- Extract image embeddings
- Train a simple linear classifier on labeled data

**Important:**

- This requires labeled training data
- It is NOT zero-shot

## 2. Zero-Shot Prediction

**Goal:** Evaluate generalization without task-specific training.

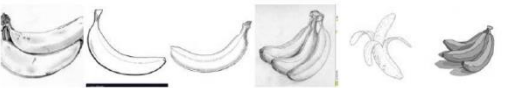
Procedure:

- Encode class names as text prompts
  - “a photo of a dog”
  - “a photo of a cat”
- Encode the image
- Compute cosine similarity
- Pick highest score

What it measures:

- Open-vocabulary capability
- Real zero-shot performance

# Zero Shot CLIP

| DATASET                                                                                                        | IMAGENET<br>RESNET101 | CLIP ViT-L |
|----------------------------------------------------------------------------------------------------------------|-----------------------|------------|
|  <p>ImageNet</p>              | 76.2%                 | 76.2%      |
|  <p>ImageNet V2</p>           | 64.3%                 | 70.1%      |
|  <p>ImageNet Rendition</p>    | 37.7%                 | 88.9%      |
|  <p>ObjectNet</p>             | 32.6%                 | 72.3%      |
|  <p>ImageNet Sketch</p>       | 25.2%                 | 60.2%      |
|  <p>ImageNet Adversarial</p> | 2.7%                  | 77.1%      |

# Question

---

Question: **Which statement about zero-shot CLIP is correct?**

- A. It requires fine-tuning on the target classes before inference
- B. It uses a fixed classifier that must be retrained for new classes
- C. It predicts classes by comparing image embeddings with text prompt embeddings
- D. It can only work on classes seen explicitly during supervised training
- E. It is the same thing as linear probing



# Objectives

---

By the end of this lecture, students should be able to:

- explain the difference between **supervised**, **self-supervised**, and **unsupervised** learning
- describe the role of **pretext tasks** and **downstream tasks** in self-supervised learning
- explain how **MAE** learns visual representations by reconstructing masked image patches
- describe how **DINO** uses a **teacher–student** framework to learn from unlabeled images
- explain what makes a model a **foundation model** and why models such as **DINOv2** and **CLIP** are considered foundation models
- describe how **CLIP** uses **contrastive learning** to align images and text
- explain the meaning of **zero-shot learning** and how CLIP enables **open-vocabulary prediction**
- distinguish between **linear probing**, **fine-tuning**, and **zero-shot prediction** as different ways to evaluate or use pretrained models



THE UNIVERSITY OF BRITISH COLUMBIA

